```
        })
      })
    },
  }
 )
}
```

The pattern is similar to the React ActionCable we did in Chapter 9, *Immediate Communication with ActionCable*. If the channel doesn't exist, we create it. When the channel receives data, we parse it, and for each concert that comes in, we send an updateConcertRemaining action that finds the relevant concert buried in whatever ScheduleDay holds it and updates it:

appendix_react/01/app/javascript/contexts/schedule_context.ts

```
case "updateConcertRemaining": {
 const allDays = Object.values(state.scheduleDays).map((day) => {
  const matchingConcert = day.concerts.find(
   (concert) => (concert.id = action.concertId)
  )
  if (!matchingConcert) {
   return day
  }
  const newConcert = {
   ...matchingConcert,
   ticketsRemaining: action.ticketsRemaining,
  }
  const newConcerts = day.concerts.map((dayConcert) => {
   if (dayConcert.id === matchingConcert.id) {
    return newConcert
   } else {
    return dayConcert
   }
  })
  return {
   ...day,
   concerts: newConcerts,
  }
 })
 const newDays = propsToMap(allDays)
 return {
  ...state,
  scheduleDays: newDays,
```